

Desafio Técnico | Revelatio Studio

Sistema simples de organização de tarefas (Task Manager)

Contexto

Ferramentas de organização pessoal fazem parte do dia a dia de muitas pessoas e equipes. Aplicações como **Todoist, Notion, Asana ou ClickUp** permitem que usuários registrem tarefas, ideias e documentos em um único lugar, ajudando na organização de atividades e no acompanhamento do que precisa ser feito.

Neste desafio, queremos simular uma versão **simplificada de uma ferramenta desse tipo**.

O objetivo é desenvolver um pequeno sistema onde um usuário possa **criar, organizar e acompanhar tarefas**, além de anexar arquivos relacionados a essas tarefas.

Mais do que avaliar apenas se a aplicação funciona, queremos entender **como você estrutura uma solução**, como organiza o código e quais decisões técnicas toma ao longo do desenvolvimento.

O mais importante para nós é entender **de que maneira você resolveu o problema**, quais escolhas técnicas fez e como estruturou sua solução.

Objetivo

Desenvolver uma aplicação simples que funcione como uma **ferramenta de organização de tarefas (Task Manager)**.

O sistema deve permitir que um usuário:

- **registrar tarefas**
- **visualizar as tarefas criadas**
- **editar, finalizar ou remover tarefas**
- **anexar arquivos ou imagens às tarefas**

A proposta é criar uma base funcional que represente um **mini sistema de gestão de tarefas**, semelhante a ferramentas utilizadas no dia a dia para organização pessoal ou de trabalho.

Escopo do sistema

O sistema deve permitir que um usuário organize suas **tasks** dentro da aplicação.

Cada tarefa pode conter, por exemplo:

- título
- descrição
- prazo ou data limite
- status (ex: **pendente**, **em andamento**, **concluída**)
- data de criação
- anexos (imagens ou documentos)

A forma como esses dados são **modelados e estruturados** fica a seu critério.

Fluxos esperados do sistema

Criar uma task

Como usuário, eu quero:

- acessar o sistema
- criar uma nova task
- informar título e descrição
- definir um prazo ou data
- anexar um arquivo ou imagem relacionado

Para que eu consiga organizar minhas atividades.

Visualizar minhas tasks

Como usuário, eu quero:

- acessar o sistema
- visualizar uma lista das tasks que criei

Para acompanhar minhas atividades.

Gerenciar tasks

Como usuário, eu quero:

- editar informações de uma task
- marcar uma task como concluída
- remover uma task

Para manter minha lista organizada.

Funcionalidades esperadas

O sistema deve permitir:

- **Registrar tarefas**
- **Visualizar as tarefas criadas**
- **Editar, finalizar ou remover tarefas**
- **Anexar arquivos ou imagens às tarefas**
- **Autenticação básica de usuários**
- **Controle de acesso baseado em perfil**

Autenticação e níveis de acesso

O sistema deve possuir um **login básico de usuários**.

Cada usuário deve possuir um **nível de acesso**.

Perfis esperados

Usuário comum

Pode:

- criar tarefas
- visualizar **apenas as tarefas que ele criou**
- editar suas próprias tarefas
- remover suas próprias tarefas

Administrador

Pode:

- visualizar **todas as tarefas do sistema**
- editar tarefas
- remover tarefas
- acompanhar as tarefas de todos os usuários

Requisitos técnicos

Backend (obrigatório)

Esperamos que exista um backend responsável por:

- persistência das tarefas
- criação, edição, listagem e remoção de tasks
- gerenciamento ou associação de arquivos anexados
- autenticação básica de usuários
- controle de acesso entre **admin e usuário comum**

A estrutura da API e arquitetura do projeto ficam livres, mas esperamos **organização e clareza no código**.

Frontend (mínimo esperado)

Esperamos pelo menos uma **interface simples** que permita:

- login no sistema
- criar tarefas
- visualizar as tarefas criadas
- interagir com o sistema

O objetivo do frontend é apenas permitir que possamos **testar e visualizar o funcionamento da aplicação**.

Uma interface mais estruturada será considerada **diferencial**, mas não é obrigatória.

Observação

Como este desafio tem foco principal em **backend**, sugerimos que você utilize ferramentas que aceleram a criação da interface, inclusive utilizando **IA**, se desejar.

Ferramentas como:

- **v0**
- **Lovable**
- **Bolt**
- ou qualquer outra de sua preferência

podem ser utilizadas para gerar rapidamente um front end funcional.

A ideia é apenas termos uma interface simples para **interagir com o sistema e testar as funcionalidades**.

Liberdade de stack

A escolha de tecnologias é **totalmente livre**.

Queremos que você utilize as ferramentas com as quais **se sente mais confortável**.

Alguns exemplos de tecnologias que podem ser utilizadas:

Backend

- Node.js
- Express
- NestJS
- Python (FastAPI, Flask, Django)
- Java (Spring Boot)
- PHP (Laravel)

Frontend

- React
- Next.js
- Vue
- Angular
- HTML + JavaScript simples

Banco de dados

- PostgreSQL
- MySQL
- SQLite
- MongoDB
- Supabase
- Firebase

- Prisma ORM
- ou qualquer outra solução equivalente

Essas são apenas sugestões — a escolha da stack fica totalmente aberta.

Uso de IA

Você pode utilizar ferramentas de **IA durante o desenvolvimento**.

Caso utilize, pedimos apenas que explique brevemente:

- quais ferramentas foram usadas
- em que partes do projeto foram utilizadas
- como essas ferramentas ajudaram no desenvolvimento

Pode ser incluído no **README** ou em um pequeno arquivo adicional.

Exemplos de ferramentas que podem ser utilizadas:

- ChatGPT
- Cursor
- GitHub Copilot
- Lovable
- v0
- Bolt
- outras ferramentas similares

A ideia não é penalizar o uso de IA, mas entender **como você utilizou essas ferramentas durante o processo**.

Diferenciais (não obrigatórios)

Os seguintes pontos podem enriquecer sua solução, mas **não são obrigatórios**:

- interface mais estruturada
- utilização de IA para criação ou melhoria da interface
- filtros ou busca de tarefas
- preview de anexos
- organização das tasks por status ou prioridade

- autenticação mais robusta
- documentação da API
- testes automatizados
- uso de Docker
- deploy da aplicação
- explicação da arquitetura escolhida

Entrega

Enviar um **repositório no Git** contendo:

- código fonte do projeto
- **README.md** com instruções para rodar a aplicação

No README você pode:

- explicar rapidamente o projeto
- comentar algumas decisões técnicas
- explicar **de que maneira utilizou (ou não) IA durante o desenvolvimento**
- adicionar qualquer observação que ache relevante para entendermos sua solução
- incluir qualquer outro ponto que você considere importante para analisarmos o projeto

Essa documentação pode ser simples — o objetivo é apenas nos ajudar a entender melhor o projeto e o raciocínio por trás da implementação.

Vídeo de, no máximo, **7 minutos**, apresentando o sistema e explicando brevemente os principais pontos da implementação. O vídeo pode ser gravado utilizando ferramentas gratuitas, como Clipchamp, OBS Studio ou Loom, ou qualquer outra plataforma de preferência do candidato. O link do vídeo deverá ser incluído no **README.md**.

O que vamos avaliar

Principalmente:

- organização do código
- estrutura do backend
- modelagem de dados
- clareza da solução
- capacidade de resolver problemas
- decisões técnicas tomadas

- **de que maneira utilizou IA e como foi esse uso**

Não esperamos uma solução perfeita.

O mais importante para nós é entender **como você pensou e estruturou a solução.**